CampusConnect: A Student Event Discovery and Team Collaboration Platform

Shallini B (241701051), Mahalakshmi D (241701028)

Department of Computer Science and Design, Rajalakshmi Engineering College, Tamil Nadu, India

IEEE-Style Student Research Paper — Draft for Academic Review

Abstract—CampusConnect is a student-focused web application concept intended to bring college event discovery, event participation, project exploration, and teammate formation into one interface. Students commonly encounter event announcements across multiple channels, while forming a suitable team may require separate searches through class groups, club networks, and personal contacts. CampusConnect addresses this fragmentation through a unified interface that presents event categories and opportunity cards, an application-status view, teammate search and compatibility indicators, and project listings. The current prototype interface was developed using React and Vite with HTML, CSS, and JavaScript. Screenshots available for this paper show the home dashboard, event discovery, application tracking, teammate discovery, and project group formation pages. The work is presented as a prototype-stage design and implementation report. The screenshots establish the presence of these interface elements but do not independently prove production database persistence, successful external event scraping, a deployed recommendation model, or end-to-end transactional behaviour. The paper describes the problem, requirements, system design, interface implementation, preliminary observations, limitations, and a validation plan.

Index Terms—CampusConnect, event discovery, student collaboration, team formation, web application, teammate recommendation, React, Vite.

I. INTRODUCTION

College students are offered opportunities through symposiums, hackathons, technical festivals, workshops, competitions, project showcases, and club activities. However, information about these opportunities may be distributed across notice boards, social-media posts, messaging groups, institutional websites, and registration forms. Students may consequently miss deadlines or spend time verifying details across multiple sources. A related challenge appears when participation requires a team: a student may have an idea but lack access to peers with complementary skills, interests, or availability.

CampusConnect is designed as a unified student opportunity platform. Its central idea is to connect the stages of discovering an opportunity, reviewing its details, applying, identifying potential collaborators, and exploring projects. Instead of treating event listings and team formation as separate services, the application places them within a common navigation and interaction model.

The current work focuses on the frontend prototype and its visible interaction structure. The available screenshots show an application dashboard, event discovery page, applications page, teammate search page, and project group formation page. React supports reusable interface components, while Vite provides the development and build environment. The implementation status of services not visible in the screenshots is treated cautiously throughout this paper.

The contributions of this work are: (1) a consolidated user journey for event discovery and collaboration; (2) an interface structure for event categories, application status, candidate filters, and project cards; (3) a proposed compatibility model that communicates possible teammate fit through weighted factors; and (4) a staged validation plan that separates demonstrated interface work from functionality requiring further verification.

II. PROBLEM STATEMENT AND MOTIVATION

The problem addressed is the fragmentation of student event information and the difficulty of finding appropriate collaborators in a timely manner. A student may need to search several channels to learn what is happening, identify eligibility and deadlines, submit an application, and then find peers for a team. This multi-channel process can create information gaps and additional coordination effort.

A. Problem Definition

How might a college-focused web application help students discover relevant events, understand participation requirements, track their applications, and connect with potential teammates through a simple and consistent interface?

B. Design Goals

- Centralise event discovery and organise opportunities by category.
- Make important event information and time-sensitive deadlines visible.
- Provide a clear view of application states such as applied, accepted, rejected, upcoming, and completed where supported by the application workflow.
- Help students search for peers using skills, academic context, role fit, and availability.
- Provide a project listing space where ideas and required collaborators can be explored.
- Keep the interface minimal and navigable rather than presenting all functions on one crowded page.

III. RELATED WORK

Recommender systems provide a useful foundation for reducing information overload by ranking items according to user preferences or item characteristics. Ricci, Rokach, and Shapira describe the broader recommender-system landscape and common design approaches [1]. In CampusConnect, event discovery could use a similar approach by considering categories, interests, eligibility, and deadlines; however, the current screenshots do not establish that a trained or deployed event recommender is operating.

Team formation is a distinct problem from recommending a single item. Lappas, Liu, and Terzi study forming teams for collaboration by considering the skills required for a task and the skills possessed by candidate members [2]. This is relevant to CampusConnect's goal of

helping students identify teammates for projects or competitions. The prototype displays compatibility factors, but these should be treated as a proposed matching framework until its data, scoring logic, and performance are verified.

Web usability is also important because the intended users may access the platform under time pressure. Nielsen's usability heuristics emphasise visibility of system status, consistency, error prevention, and recognition rather than recall [3]. The application's navigation, filters, status labels, and visible calls-to-action are consistent with these general principles. A formal usability evaluation remains future work.

The references above inform the design rationale; they are not evidence that CampusConnect has achieved a particular accuracy, adoption rate, or measurable improvement.

IV. SYSTEM REQUIREMENTS

A. Functional Requirements

ID	Requirement	Prototype evidence / status
FR-1	Display a home dashboard and navigation to major modules.	Visible in supplied dashboard screenshot.
FR-2	Browse event opportunities and categories.	Event cards, category labels, and action controls are visible.
FR-3	Review application records and status categories.	Applications page and status filters are visible.
FR-4	Search and filter potential teammates.	Search/filter controls and candidate cards are visible.
FR-5	Browse project ideas and initiate project creation or teammate discovery.	Project cards and calls-to-action are visible.
FR-6	Submit and track applications end-to-end.	Requires functional testing; screenshot alone is insufficient.
FR-7	Accept or reject teammate join requests.	Requested feature; implementation requires verification.
FR-8	Import or scrape events from external sources such as KnowAFest.	Integration success is not established by the screenshots.

B. Non-Functional Requirements

- Usability: labels, navigation, and status indicators should be understandable to first-time student users.
- Maintainability: reusable React components should make event cards, filters, and navigation easier to update.
- Responsiveness: the layout should remain usable on common desktop and mobile screen sizes.
- Data quality: event dates, organiser details, eligibility, and registration links should be checked before publication.
- Privacy and security: student profiles, resumes, skills, and contact information require access control and consent.
- Reliability: application status and join-request decisions should be persisted and displayed consistently when backend services are connected.

V. PROPOSED SYSTEM ARCHITECTURE

At the interface level, CampusConnect is organised into a React-based presentation layer containing dashboard, event discovery, applications, teammate discovery, and project pages. User actions such as searching, filtering, opening an event, or selecting a project should be handled by the relevant page components. In a complete deployment, these components would communicate with application services for authentication, event data, applications, profiles, resumes, matching, notifications, and project membership.

Layer	Responsibilities	Status in this report
Presentation layer	React pages, navigation, cards, search and filters.	Visible prototype UI.
Application/service layer	Validate applications, handle join requests, compute match scores, manage event updates.	Design requirement; verify implementation.
Data layer	Store users, event listings, applications, skills, resumes, projects and team memberships.	Database not confirmed from supplied screenshots.
External integrations	Retrieve event metadata from trusted event sources and preserve source links.	KnowAFest integration proposed; scraping success unverified.
Security layer	Authentication, authorisation, validation, privacy and safe file handling.	Required for production; needs dedicated testing.

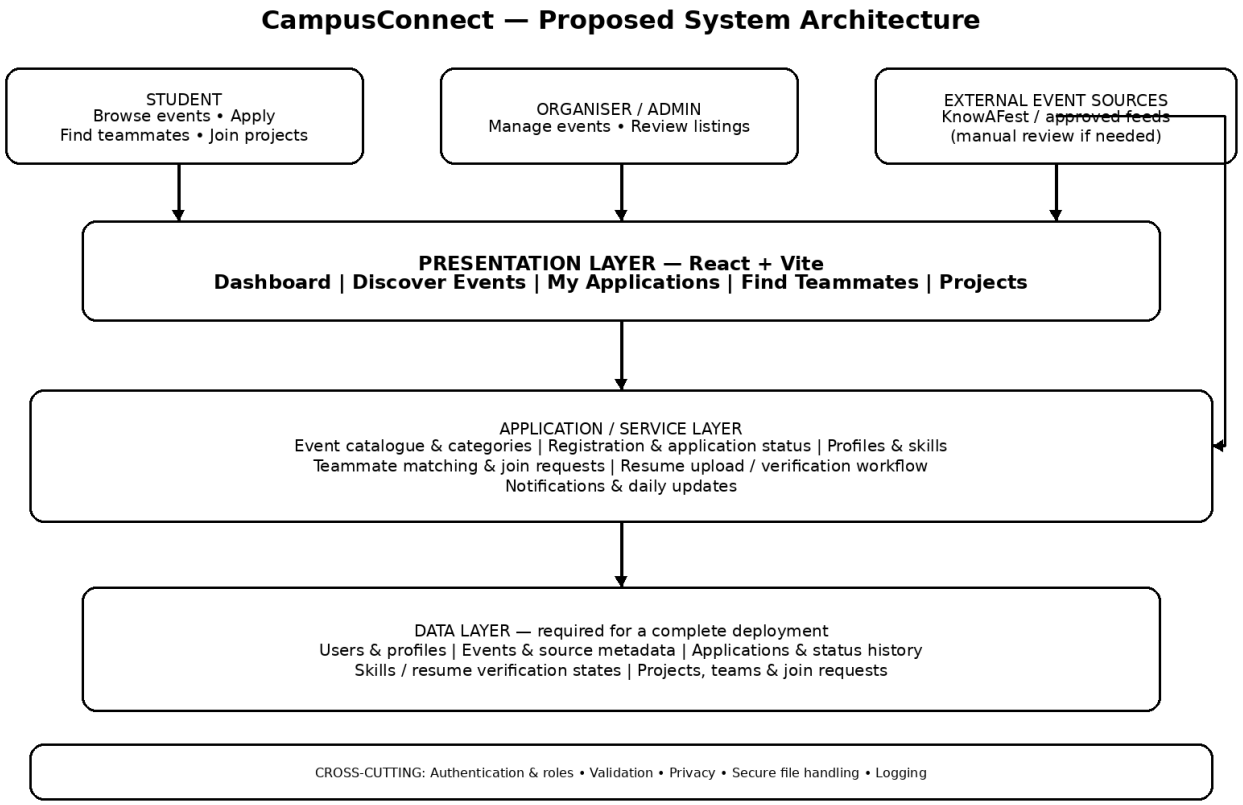


Fig. 1. Proposed logical architecture of CampusConnect. The diagram represents the intended component boundaries; it does not imply that all services, storage, or external integrations are currently implemented.

VI. METHODOLOGY AND INTERFACE DESIGN

A. Development Approach

The project follows an iterative prototype approach: identify student pain points, organise the main modules, implement the page structure, review the screens, and refine the interface. The supplied screenshots are used as evidence of the current visual implementation. This paper does not assume that every proposed action has been tested against a live backend.

B. Event Discovery

The Discover Events page presents opportunities as cards and uses visual labels to distinguish event types and highlight deadlines. Search and feed-synchronisation controls are present in the screenshot. A complete implementation should show the organiser, event date, location or online mode, eligibility, cost, deadline, team-size rules, source link, and the status of an in-app application. If an external registration page is required, the interface should clearly identify that transition.

C. Application Tracking

The My Applications screen groups records using status-oriented tabs and includes search. Cards show event-related information and actions such as viewing an event or withdrawing where applicable. The interface design aims to reduce the need to remember which external form was used. Correctness depends on testing whether a submission actually changes the stored application state and whether the resulting state is shown consistently.

D. Teammate Discovery and Matching

The Find Teammates page presents candidate cards and controls for searching or filtering by attributes such as department, year, skills, or role fit. The visible interface also displays a weighted compatibility breakdown: skills (40%), interests (20%), availability (15%), experience (10%), resume verification (10%), and department match (5%). These weights sum to 100%. They should be described as the prototype's displayed matching design, not as validated evidence of prediction quality.

A possible weighted score can be expressed as $M(u,v) = 0.40S + 0.20I + 0.15A + 0.10E + 0.10R + 0.05D$, where each factor is normalised to the range 0–1 for two users u and v. This equation documents the visible factor weighting concept; actual computation and normalisation must be confirmed in source code before it can be claimed as implemented. Resume verification should mean a clearly defined review or validation procedure, not merely the presence of an uploaded file.

E. Project Group Formation

The project listing page presents project ideas with domain or technology context and provides actions to create a project or discover teammates. This design supports students who have an idea but need collaborators. Further development should include project-owner controls, required roles, team capacity, join-request status, and explicit acceptance or rejection actions.

VII. TECHNOLOGY STACK

Technology	Role in the project	Evidence / qualification
React	Component-based frontend and page composition.	Confirmed by the project description; source-code audit not performed in this paper.
Vite	Frontend development server and build tooling.	Confirmed by the project description.
HTML	Semantic structure of the web interface.	Core web technology used with the frontend.
CSS	Layout, typography, colours, cards, spacing and responsive styling.	Visual styling is evident in screenshots.
JavaScript	Client-side interactions and application logic.	Core frontend technology; individual workflows require testing.
External event source (KnowAFest)	Potential source of event information.	Integration was requested, but successful scraping or API access is not verified.

No backend framework, database, authentication provider, or AI model is named as confirmed because the available evidence does not establish which service is currently connected. These should be added to the paper only after checking the project source and deployment configuration.

VIII. PRELIMINARY IMPLEMENTATION OBSERVATIONS

The supplied screenshots demonstrate a coherent multi-page interface with consistent navigation and a shared visual identity. The observed screens correspond to five central user tasks: entering the dashboard, browsing events, reviewing applications, searching for teammates, and exploring project groups. The interface separates these tasks into pages rather than forcing all content into one screen.

Observed interface	What can be stated from screenshot	What cannot be concluded from screenshot alone
Home dashboard	Dashboard layout and opportunity sections are visible.	Live updates, real-time counts, or backend data freshness.
Discover Events	Event cards, categories, deadline emphasis and action controls are visible.	Successful event scraping, accurate live listings, or successful registration.
My Applications	Status categories and application cards are visible.	Persistent storage, actual acceptance workflow, or successful withdrawal.
Find Teammates	Candidate cards, search/filter controls and match labels are visible.	Accuracy of matching, verified resumes, or real join-request notifications.
Projects	Project listings and team-oriented actions are visible.	Creation persistence, membership permissions, or completed collaboration workflow.

The evidence supports describing the work as a partially implemented frontend prototype. It does not support quantitative performance claims, such as recommendation accuracy, number of successful registrations, user satisfaction percentage, or system response time.

IX. TESTING AND EVALUATION PLAN

A structured test plan is required before the prototype can be reported as a fully functional application. Testing should be performed on the actual running build and results should be recorded with date, environment, test data, expected outcome, observed outcome, and pass/fail status.

Test ID	Test case	Expected result
T-01	Open each main navigation item.	Correct page loads and active navigation state is clear.
T-02	Search events using a title or keyword.	Relevant matching events are shown; empty results are handled clearly.
T-03	Apply category and deadline filters.	Displayed event list reflects selected filters.
T-04	Open an event and start an application.	Correct event details and registration flow are presented.
T-05	Submit an application and revisit My Applications.	The application appears with a correct status after reload.
T-06	Search and filter teammate candidates.	Results follow the selected query and filters.
T-07	Send a teammate join request; accept or reject it from the recipient account.	Both users see the correct request status.
T-08	Create a project and request to join it.	Project and membership changes persist correctly.
T-09	Import external event information.	Source, date, URL and event details are validated; errors are logged.
T-10	Upload a resume file.	Allowed file types and sizes are enforced; access is restricted appropriately.
T-11	Check layout at desktop and mobile widths.	No critical overlap; navigation and controls remain usable.
T-12	Test invalid input and service failure.	Clear error messages appear and no inconsistent

		status is saved.

No pass/fail results are reported here because execution logs or test evidence were not supplied. This table is a proposed validation plan and should not be presented as completed testing.

X. DISCUSSION

CampusConnect's intended value lies in connecting opportunity discovery and collaboration in one workflow. A student may begin with an event, identify the skills needed for participation, find suitable peers, and then coordinate through a project or team listing. The approach may reduce the effort of switching between disconnected sources, but that benefit must be evaluated through user studies rather than assumed.

The interface also creates responsibilities. Event information must be accurate and current; external event pages should be linked transparently; candidate recommendations should be explainable; and resume or profile information must not be exposed without permission. Compatibility scores should not be treated as objective judgments of a student's ability. Users should be able to inspect the factors behind a recommendation and make their own decisions.

The strongest next step is not to add more visible modules alone, but to validate the core end-to-end journeys. Reliable persistence, correct state transitions, clear error handling, and trustworthy event data are necessary for the application to move from a visual prototype to a dependable campus service.

XI. LIMITATIONS AND FUTURE WORK

A. Current Limitations

- The available evidence consists primarily of interface screenshots and project-context descriptions, not a complete source-code audit.
- Backend, database, authentication, resume verification, and notification behaviour are not independently confirmed.
- External event ingestion from KnowAFest is a requested capability; successful access, scraping permission, and update reliability have not been demonstrated.
- The displayed teammate compatibility weights have not been evaluated against ground-truth team outcomes.
- No formal usability study, benchmark, or quantitative system test results were provided.

B. Future Work

- Connect and verify a persistent data store for users, events, applications, projects and join requests.
- Implement authenticated roles for students, organisers and administrators, with approval and moderation controls.
- Use permitted APIs or documented feeds where available; otherwise provide verified event submission and manual review rather than relying on brittle scraping.
- Define an explainable matching algorithm, test it with consented profile data, and allow users to adjust preferences.
- Implement secure resume upload, explicit verification states, file validation, and restricted access.
- Add notifications for deadlines, application status changes, and join-request decisions.
- Run usability testing with students and administrators and report measured findings transparently.

XII. CONCLUSION

This paper presented CampusConnect, a student-focused platform concept for connecting college event discovery, application tracking, teammate discovery, and project group formation. The current React and Vite interface includes visible pages for the dashboard, event discovery, applications, teammate search, and projects. The design's central contribution is an integrated journey from discovering an opportunity to finding collaborators. The available screenshots support claims about the interface structure, but not about unverified backend services, successful external scraping, trained AI, or measured outcomes. The next phase should prioritise end-to-end functional validation, trusted event data, secure profile handling, and evaluation with real student users.

REFERENCES

[1] F. Ricci, L. Rokach, and B. Shapira, Eds., Recommender Systems Handbook, 3rd ed. New York, NY, USA: Springer, 2022.

[2] T. Lappas, K. Liu, and E. Terzi, "Finding a team of experts in social networks," in Proc. 15th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD '09), Paris, France, 2009, pp. 467–476.

[3] J. Nielsen, "10 usability heuristics for user interface design," Nielsen Norman Group. [Online]. Available: https://www.nngroup.com/articles/ten-usability-heuristics/. Accessed: Oct. 15, 2026.

[4] React Team, "React documentation." [Online]. Available: https://react.dev/. Accessed: Oct. 15, 2026.

[5] Vite Team, "Vite documentation." [Online]. Available: https://vite.dev/. Accessed: Oct. 15, 2026.